Predicting Hotel Booking Likelihood

A Learning-to-Rank Approach on the Expedia Dataset

VU Data Mining Techniques 2026, Assignment 2

Group VU-DM-2026-Group-91

Author placeholders: <Name 1>, <Name 2>, <Name 3>

Contents

1. Introduction

Search ranking is the algorithmic heart of every modern online travel platform. When a user types a destination, dates, and party size into Expedia, the website returns a list of hotel properties ordered by their estimated relevance to that user. Getting the order right is commercially crucial: a hotel placed first is many times more likely to be booked than one shown twentieth. Conversely, surfacing the wrong property at the top is a lost sale that the booking funnel rarely recovers from.

This assignment asks us to build such a ranker. We are given approximately five million rows of Expedia search data spanning November 2012 to June 2013. Each row describes one displayed hotel within one search query, together with the click and booking outcomes. The competition objective is to predict, for the held-out test set, the order in which hotels should be shown so that booked hotels appear near the top. Performance is measured with NDCG at rank 5 (NDCG@5) averaged across all search queries.

We treated the problem as a learning-to-rank task. We built two models: a Ridge regression baseline that gives us a 'what does a simple model do' reference, and a LambdaMART ranker (LightGBM Ranker) as the main approach. The choice of LambdaMART is grounded in Lecture 7 of the course, which presents learning-to-rank as the algorithmic backbone of modern recommender systems. The original Expedia production ranker is itself a LambdaMART variant, so we are literally re-building the algorithmic idea on the data it produced.

This report follows the CRISP-DM process model from the assignment brief. Section 2 is a glossary of the technical jargon we use throughout. Section 3 covers Related Work. Sections 4 to 8 cover the five CRISP-DM stages from Data Understanding to Bias Investigation. Section 9 reflects on what we learned. Throughout, we cross-reference the cell numbers in our notebook so the code provenance of each number is traceable.

Headline result: our final public-leaderboard Kaggle score was NDCG@5 = 0.38694, achieved with LambdaMART using 45 engineered features and 392 boosting iterations. We discuss this number in context in Section 7 and Section 10.

2. Glossary

Before we dive in, here are the technical terms that appear most often. Each term is defined in the way we use it in this report and explained in plain language so a reader with no machine-learning background can follow along.

Search query (srch_id)

One user search session. Each search returns a list of roughly 25 hotel options. Our dataset contains 199,795 unique searches in the training set and 332,787 in the test set. Each row of the dataset corresponds to ONE hotel within ONE search.

Property (prop_id)

One hotel in the catalogue. There are 129,113 unique hotels in the training set.

Relevance grade

A 3-level label we constructed from the raw click and booking indicators. A hotel that the user booked gets a relevance of 5. A hotel that the user clicked on but did not book gets 1. A hotel the user ignored gets 0. The ranking model is trained against this grade.

NDCG@5 (Normalized Discounted Cumulative Gain at rank 5)

The official evaluation metric for this competition. For one search, we look at the top-5 hotels the model

recommended (in that order), convert their relevance grades using the gain formula (2^rel - 1) / log2(rank + 1), and sum across the five positions. We then divide by the best possible score for that search (the IDCG) so the result is between 0 and 1. NDCG@5 = 1 means the model ranked the booked hotel first; 0 means none of the top-5 was clicked or booked.

Learning-to-rank (LTR)

A class of machine-learning techniques that learn to order a list of items, rather than predict a number for each item independently. The model is trained on listwise comparisons (which items in the list should be above which) rather than pointwise scores.

LambdaMART

The most popular learning-to-rank algorithm. It is a gradient-boosted decision-tree forest trained with a loss function whose gradients are weighted by the NDCG-swap penalty for each pair of items in a search. The 'lambdas' in the name are those NDCG-swap weights. Implemented in LightGBM as objective='lambdarank'.

LightGBM

Microsoft's open-source gradient-boosted-tree library. We use it for the main LambdaMART model. It handles categorical features natively (via subset-split criterion) and trains fast even on millions of rows.

Group-aware split

A train/validation split that keeps all rows of the same search query together. Required for ranking tasks: a naive random row-level split would leak the booking signal across the split boundary.

Ridge regression

A linear regression that adds an L2 penalty on the coefficient sizes. Behaves well when several input features are correlated. We use it as the baseline model.

Per-search relative feature

A feature that describes a hotel relative to the OTHER hotels in the same search. Example: 'this hotel is the 3rd cheapest of the 25 shown'. Such features capture the listwise structure of the ranking task and consistently rank among the most predictive features.

Demographic parity

A fairness metric (Lecture 8). Two groups are demographically equal under a model if the model produces positive outcomes for them at the same rate. In our ranking setting, the analogue is 'equal share of top-5 slots across two groups'.

Re-weighting (pre-processing mitigation)

A fairness technique (Lecture 8) where rows of an under-served group are given higher training weights so the model 'cares more' about getting them right. We applied this in Section 8.

Kleinberg impossibility

A result from Lecture 8 stating that several common definitions of fairness (calibration, error-rate balance, demographic parity) cannot all hold simultaneously except in trivial cases. Practical work on fair models therefore involves choosing which definition to prioritise.

Cold-start

The situation where the system has to make a prediction for an item (or user) it has never seen during training. In our case: hotels that appear in the test set but never in the training set.

3. Related Work

Note: this section is the responsibility of teammate X in our group and will be expanded in the final draft with proper citations to the ICDM 2013 papers. The summary below is the working draft.

The dataset we use is a near-replica of the data released for the 2013 ICDM Personalize Expedia Hotel Searches Kaggle competition. The original competition produced a body of public solution write-ups that we drew on for guidance, particularly around which features tend to be most predictive. Across the published top-ten solutions, four feature families recur:

- Per-search relative features (rank or gap of a hotel's price, star rating, or location score within its search). Multiple top finishers reported these as their highest-importance features overall.
- Per-property historical aggregates (average booking rate, average price, std of price) computed across the full training set. These act as a global 'reputation' signal that complements within-search features.
- The prop_location_score2 column, an internal Expedia desirability score, was repeatedly named as one of the top three single features.
- Position bias features. The random_bool column tells us whether the user saw a randomly ordered list, which lets the model correct for the way the production ranker shaped historical click and booking outcomes.

The best published solutions reached NDCG@5 of around 0.54, using gradient-boosted-tree ensembles with 150+ features. Our work re-implements the same core idea with a smaller feature set (45 features) and a single LambdaMART model rather than an ensemble. Section 7 compares our number to these published numbers.

4. Data Understanding (Task 2)

4.1 The dataset at a glance

The training set is a 1.18 GB CSV with 4,958,347 rows across 54 columns. Each row represents one search query x hotel pair: the user's search context (destination, dates, party size, country of origin), the displayed hotel's attributes (star rating, review score, price for this search, internal location scores, brand affiliation), 24 competitor-price columns describing how Expedia's price compared to eight other booking sites at search time, and the labels click_bool and booking_bool. The training set spans 199,795 unique searches over the eight-month window.

The test set is structurally similar but withholds the three training-only columns (position, click_bool, booking_bool). There is no overlap of search IDs between the two sets, and many but not all property IDs in the test set also appear in training.

4.2 Loading the data efficiently

Pandas' default datatypes (int64, float64) consume more memory than the data actually requires. By specifying smaller datatypes upfront, we cut the in-memory footprint of the training set from approximately 2 GB to 832 MB. Specifically, 0/1 columns are stored as int8 (one byte each), country and site IDs as int16, hotel and search IDs as int32, and continuous features as float32. The full datatype dictionary lives in Cell 2 of the notebook. This step is the 'data reduction by type narrowing' idea from Lecture 2 and pays back later: every groupby and merge on the table runs measurably faster.

4.3 Missing values

A column-by-column count of missing values revealed five problematic blocks (Cell 3 of the notebook).

Each block has its own root cause, which informs the imputation strategy in Section 5.2.

Column block	Missing %	Likely cause
visitor_hist_starrating, _adr_usd	~95%	Anonymous / new visitor
srch_query_affinity_score	~93%	No internet-search match
comp1..8 (24 columns)	60-97%	No live competitor quote at search time
orig_destination_distance	~32%	Origin city not resolved
prop_location_score2	~22%	Score not computable for that property
prop_review_score	~0.15%	New hotel with no reviews yet

4.4 Class imbalance and the NDCG@5 relevance grade

Of the 4.96 million training rows, only 2.79% have booking_bool = 1 and 4.49% have click_bool = 1. This means the median search shows the user roughly 25 hotels, of which one is clicked and 0 or 1 is booked. Predicting 'no booking' for every row would therefore be 97% accurate - a fact which makes accuracy a useless evaluation metric for this task. NDCG@5, by contrast, directly rewards a ranker for placing high-relevance hotels near the top of each search.

We construct the 3-level relevance grade in Cell 4: 5 if booked, 1 if clicked-only, 0 otherwise. This is the column that NDCG@5, and ultimately our LambdaMART loss function, operates on.

4.5 Data quality finding: the $19.7M price

Cell 6 plots distributions for the six numerical features we expect to drive the model. The most striking finding is that price_usd has a maximum value of $19,726,328 - clearly a corrupted record, since real hotel prices for a single night essentially never exceed a few thousand dollars. The 99th percentile sits at $599 and the 99.9th at $2,060. Left unchanged, this single outlier (and a handful of similar values in the long tail) would dominate any model's loss function. We handle it in Section 5.1 with a 99.9-percentile clip plus a log transform.

4.6 Position bias

A specific Lecture 8 concern is position bias: users click more on hotels shown near the top of the page than on hotels lower down, partly because they look more, partly because the ranker shows better hotels first. The random_bool column lets us measure the two effects separately. For searches where random_bool = 0 (normal Expedia ranking), mean relevance falls steeply from position 1 to position 40. For random_bool = 1 (random ordering), mean relevance is essentially flat. The gap between the two curves is the visual position-bias effect, and the steepness of the random_bool = 0 curve is what the ranker is exploiting. This finding directly motivates the bias investigation in Section 8.

4.7 Iterative EDA: which features predict booking

A first-pass EDA covered data quality. A second pass, motivated by the assignment's explicit guidance that EDA is iterative, focused on which features actually correlate with booking. We computed booking rate per decile for the three features we considered most predictive (Cell 7a):

Feature	Booking-rate range across deciles	Strength
prop_starrating (0-5)	1.33% to 3.32% (2.5x)	Moderate
price_usd decile	1.41% to 3.37%	Moderate, non-monotonic
prop_location_score2	1.09% to 5.33% (4.9x)	Strongest of the three

Two things follow. First, prop_location_score2 is empirically the single strongest predictor of booking probability, confirming what the published ICDM 2013 solutions reported. Second, the absolute price_usd

decile shows a NON-MONOTONIC booking-rate curve. That is the empirical signal that motivates engineering RELATIVE price features in Section 5.4: a $200 hotel is expensive in a list of $80 hostels and cheap in a list of luxury resorts; only the relative position carries useful information.

We also measured the long-tail of property popularity (Cell 7c). The training set contains 129,113 unique hotels. 66.4% of these were never booked at all during the eight-month window, and the top 1% of most-booked properties account for 23.1% of all bookings. This is a textbook long-tail RecSys distribution (Lecture 7) and motivates the per-property aggregate features in Section 5.5 as a partial answer to the cold-start problem.

5. Data Preparation (Task 3)

This section walks through the feature-engineering pipeline that turns the raw CSV into a 45-column feature matrix the model can consume. Eight sub-sections; each ends as a numeric column or columns on the training DataFrame.

5.1 Price outliers (Cell 8)

The $19.7M price outlier described in Section 4.5 makes the raw price_usd column unsafe to use directly. We construct two derived columns:

- price_usd_log = log(1 + price_usd). The +1 protects against zero prices (rare but present). The transformed distribution is approximately normal and is what we feed to the Ridge baseline.
- price_usd_clipped = price_usd capped at its 99.9th percentile ($2,060.04). Used for tree-based modelling and as the basis for the per-search relative features in Section 5.4.

The 99.9-percentile threshold is stored as a constant (PRICE_CLIP_THRESHOLD) so the same value is re-applied to the test set during inference. This guarantees train and test see exactly the same preprocessing rule - a critical detail for avoiding silent train/test mismatch bugs.

5.2 Missing-value handling (Cell 9 + 9b)

Lecture 2 distinguishes between missing AT random (impute with a central tendency) and missing NOT at random (use a sentinel value plus an indicator flag). We handle each of our five missing-value blocks according to its mechanism:

Block	Strategy
visitor_hist_* (95% NaN)	Sentinel -1 + has_visitor_history flag
prop_location_score2 (22%)	Per-destination median, fallback global
orig_destination_distance	Per-destination median, fallback global
prop_review_score (0.15%)	Global median
srch_query_affinity_score	Sentinel min-1 + has_query_affinity flag
comp1..8 (24 columns)	Fill 0; collapsed to summaries in 5.6

A subtle bug surfaced here. Our initial implementation computed the indicator flags AFTER the fillna in the same cell, which made the cell non-idempotent: running it a second time on already-filled data produced indicators that were uniformly true rather than the intended 5% / 7% prevalence. We caught this during evaluation and added Cell 9b, a defensive reconstruction cell that derives the indicators from the imputed columns by detecting the sentinel value. The lesson logged in the process report: data-preparation cells need to be designed for re-execution from the start.

5.3 Time features (Cell 10)

Cell 10 extracts four datetime features from the parsed date_time column: search_hour (0-23), search_dayofweek (0-6), search_month (1-12), and search_is_weekend. The fourth is a convenience derivation for the linear baseline, which cannot synthesise 'weekend' from dayofweek alone without one-hot encoding. Cell 7b had already told us day-of-week is weak (booking rate spread of only 0.08 percentage points across all seven days), but the columns cost almost no memory and may help LightGBM in interactions.

5.4 Per-search relative features (Cells 11-12)

This is the highest-leverage feature-engineering step in the entire pipeline. The assignment brief hints at it

directly: 'you might want to compare the different properties that resulted from the search instead of learning from them one by one.' Cell 7a confirmed empirically that absolute price was non-monotonic in booking rate; the rank of a price within its own search is the listwise comparison signal a learning-to-rank model really needs.

We built six per-search rank features:

- price_usd_rank_in_search: the hotel's price rank within its search, normalised to [0, 1].
- price_usd_gap_vs_search_median: signed difference between this hotel's price and the median price in the same search.
- price_usd_zscore_in_search: how extreme this price is relative to its own search's spread.
- prop_starrating_rank_in_search, prop_location_score2_rank_in_search, prop_review_score_rank_in_search: rank versions of the three quality features.

These six features turned out to be five of the model's top ten features by gain importance (Section 7.3). The 30 minutes spent designing them was the highest-ROI half-hour of the project.

5.5 Per-property aggregates (Cell 13)

Cell 13 computes four cross-search statistics per prop_id: prop_id_count (how many times this hotel appeared), prop_id_price_mean, prop_id_price_std, and prop_id_location_score2_mean. These are merged back into every training row by prop_id and act as a 'global reputation' signal that complements the within-search features. For hotels in the test set that never appeared in training (the cold-start case from Lecture 7), the merge produces NaN; we fill with the global mean of each aggregate, treated as the 'average hotel' default.

5.6 Competitor summary features (Cell 14 + 14b)

The 24 raw comp{N}_* columns are too sparse for trees to use effectively. Cell 14 collapses them into five summary features:

- comp_num_cheaper: number of competitors Expedia is cheaper than.
- comp_num_more_expensive: number Expedia is more expensive than.
- comp_net_rate: sum of all 8 comp_rate values (range -8 to +8).
- comp_mean_rate_pct_diff: average absolute percent-difference across the 8 competitors.
- comp_num_inv_advantage: number of competitors that don't stock this hotel.

Cell 14's first output flagged a data-quality problem: comp_mean_rate_pct_diff had a maximum of 125,198 (i.e. 125,000% rate difference). The assignment defines the column as |Expedia - competitor| / Expedia * 100, so values above a few hundred imply a corrupted source row. Cell 14b clips the column at the 99th percentile of its non-zero values (threshold = 25.50). Only 0.32% of rows were affected, but left unclipped that single outlier would have dominated any tree's variance-based split criterion for this feature.

5.7 Train / validation split (Cell 15)

The assignment explicitly requires us to split the data to validate our approach before generating Kaggle predictions. A naive row-level random split would leak the target signal across the split boundary (see Glossary: Group-aware split). We use sklearn's GroupShuffleSplit with groups = srch_id, test_size = 0.2, and random_state = 42. The result:

Split	Searches	Rows
Train	159,836 (80.0%)	3,966,682
Val	39,959 (20.0%)	991,665

Two sanity checks pass: zero search-ID overlap between the splits, and a booking-rate difference of 0.019 percentage points (target: < 0.1pp). The split is correct and reproducible.

5.8 Final feature matrix and pickle checkpoint (Cell 16)

Cell 16 closes Task 3 with three artefacts saved to disk: the full feature DataFrame as a pickle, a preprocessing bundle with the saved thresholds and per-property aggregates lookup, and the split indices as a compact .npz file. This means Task 4 can start with a single pd.read_pickle(...) call instead of re-running the full data preparation pipeline.

The final matrix has 45 features. Seven are declared categorical (site_id, three country IDs, destination ID, and the three time integers); LightGBM will handle these with its native subset-split criterion rather than treating them as ordinal numbers. Eight columns are explicitly excluded - the three labels, the training-only position and gross_bookings_usd, the raw date_time replaced by Section 5.3 features, prop_id whose signal is captured by Section 5.5 aggregates, and srch_id which is the group key.

6. Modelling (Task 4): Algorithm Explanations

6.1 Why we trained two models

A baseline serves two purposes. First, it tells us how much the more sophisticated model is actually contributing - the comparison can then be cited explicitly in the report. Second, it acts as a sanity check on the data pipeline: a linear model that scores below random would indicate features or labels are mis-aligned, and one that scores unrealistically well would indicate target leakage. The baseline catches both failure modes early.

6.2 The Ridge regression baseline (Cell 18)

Ridge regression is a linear regression with an L2 penalty on the coefficient sizes. We chose Ridge over plain LinearRegression because our feature matrix has correlated columns by construction (e.g. price_usd_log and price_usd_clipped are two encodings of the same underlying price), and Ridge handles such multicollinearity gracefully.

We standardise the numeric features before fitting using StandardScaler so the L2 penalty does not preferentially penalise features with large raw magnitudes. The scaler is fitted on training ONLY and applied to validation - the correct non-leaky protocol.

We exclude the seven categorical features from the baseline because a linear model cannot use a high-cardinality categorical column directly (one-hot encoding would create millions of new columns for srch_destination_id, and target encoding has leakage risk). LightGBM handles them natively, so the categoricals go in there.

The baseline target is the same NDCG relevance grade (5, 1, 0). The Ridge model predicts a continuous score, which we use purely to RANK hotels within each search.

Baseline result: NDCG@5 = 0.32414 on the validation split.

6.3 LambdaMART, in plain language

LambdaMART is built on three ideas stacked on top of each other:

- DECISION TREE: a small flowchart of if-then-else questions on the features. Example: 'Is the hotel a chain? If yes, look at price; if no, look at review score.' Each leaf of the tree gives a numeric score.
- GRADIENT BOOSTING: a forest of many small decision trees, where each new tree is trained to correct the errors of all previous trees. Trees are added one at a time. After 392 trees, our model stops because the validation NDCG@5 has stopped improving.
- LAMBDARANK: instead of training each tree on a regression loss (predict the score directly), each tree is trained on a LISTWISE loss whose gradients (the 'lambdas') are weighted by how much the NDCG@5 would change if we swapped each pair of items in a search. The model therefore optimises directly for the metric we care about.

LightGBM (Microsoft's open-source library) implements LambdaMART via objective = 'lambdarank' with metric = 'ndcg' and eval_at = [5]. We use it because it is fast (1.1 minutes to train on our 4M rows), it handles categorical features natively (important for our 50,000+ destination IDs), and it has a clean callback API for early stopping.

6.4 Hyperparameters (Cell 19)

Parameter	Value	Why

learning_rate	0.05	Standard for ranking; converges in ~500 trees
num_leaves	63	Slightly above default 31 for richer interactions
min_child_samples	50	Prevents tiny leaves on imbalanced data
feature_fraction	0.9	Stochastic regularisation: 90% of features per tree
bagging_fraction	0.9	90% of rows per iteration
num_boost_round	1000	Upper cap; early stopping cuts us off earlier
early_stopping	50	Stop when val NDCG@5 hasn't improved for 50 rounds

Early stopping fired at iteration 392 (well before the 1000-round cap). Training NDCG@5 was 0.559 at that point, validation NDCG@5 was 0.38601 - a 17-percentage-point gap that indicates real but bounded overfitting, with the early-stopping callback catching the right moment to stop.

6.5 Group information

The single most important implementation detail in LambdaMART training is the 'group' argument to lgb.Dataset. It is an array giving the row count of each search in the dataset, in the order the rows appear (e.g. [25, 30, 18, ...] = first search had 25 hotels, second had 30, etc.). LightGBM uses this to know which rows belong together when computing pairwise NDCG-swap gradients. Without the group argument, the model would treat every row as belonging to its own one-row search and the ranking objective would degenerate.

We construct the group array via train.iloc[train_idx].groupby('srch_id', sort=False).size(). Because GroupShuffleSplit preserves row order, this groupby produces sizes in exactly the right sequence.

7. Results

7.1 NDCG@5 verification (Cell 20)

LightGBM reports its own NDCG@5 during training. We cross-checked it against our hand-written ndcg_at_5_per_query helper to make sure the number we are quoting is correct. The two implementations agreed to 0.00000 - zero difference at five decimal places. This eliminates any concern that off-by-one or zero-IDCG mishandling is inflating the score.

7.2 Headline comparison: Ridge vs LambdaMART

Model	Features	Best iters	Val NDCG@5	Lift vs baseline
Ridge baseline	38 (numeric only)	n/a	0.32414	+0.000
LambdaMART	45 (incl 7 categorical)	392	0.38601	+0.062

The 6.2-percentage-point lift is real and meaningful for a ranking task. For context: a random shuffle would score about 0.20, the Ridge baseline lifts us by 0.12 over random, and LambdaMART adds another 0.06 on top.

7.3 Feature importance (Cell 21)

The LambdaMART feature importance, sorted by total gain across all splits, tells a clean story:

Rank	Feature	Gain %	Origin
1	srch_destination_id	64.9%	Raw (categorical)
2	prop_location_score2_rank_in_search	7.0%	Engineered (5.4)
3	prop_starrating_rank_in_search	3.5%	Engineered (5.4)
4	price_usd_zscore_in_search	3.0%	Engineered (5.4)
5	prop_location_score2	2.6%	Raw
6	prop_location_score1	1.9%	Raw
7	prop_id_price_mean	1.9%	Engineered (5.5)
8	price_usd_log	1.9%	Engineered (5.1)
9	price_usd_rank_in_search	1.5%	Engineered (5.4)
10	promotion_flag	1.4%	Raw

Two interpretive points follow.

First, srch_destination_id dominates with 65% of total gain. LightGBM's native categorical handling lets it memorise destination-specific booking patterns very efficiently: users searching for Amsterdam, Bali, or New York have characteristically different preference distributions and the model exploits this directly. This is also a likely contributor to the 17-point training/validation gap, since destinations are not infinitely repeated.

Second, four of the top ten features are the per-search relative features we engineered in Section 5.4. This is strong empirical validation of the listwise-comparison design choice.

7.4 Kaggle public-leaderboard score (Cell 22)

We applied the same preprocessing chain (Section 5) to the test set, predicted scores with the trained ranker, and wrote a submission.csv with the required srch_id, prop_id columns sorted by predicted score descending within each search. The cold-start cases (test hotels never seen in training) are handled by

filling per-property aggregates with the global means.

The public-leaderboard score was NDCG@5 = 0.38694, within 0.001 of the local validation score of 0.38601. This near-perfect transfer demonstrates that our GroupShuffleSplit validation does what it is meant to: hold out searches in a way that genuinely simulates the Kaggle held-out set.

For comparison: random shuffle approx 0.20, our Ridge baseline 0.32414, our LambdaMART 0.38694, published ICDM 2013 winners approx 0.54.

8. Bias Investigation (Task 5)

8.1 Choice of bias axis

The assignment text explicitly names 'underrepresentation of certain hotels' as a possible bias to investigate. The cleanest axis to operationalise this on is prop_brand_bool - whether a hotel is part of a major chain (1) or independent (0). Chain hotels typically have more reviews, more uniform service, and richer booking history, meaning our model has more reliable signal to score them confidently. Independent hotels might therefore be systematically under-ranked even when they would actually have been the right booking choice.

Cell 23 establishes the population baseline. In the training data, 63.5% of rows are chain hotels and 36.5% are independent. Chains are booked at a 2.92% rate when displayed, versus 2.58% for independents (a 13% relative gap). Mean prices are nearly identical, so the groups are not separable by price alone.

A FAIR model under demographic parity would surface roughly 63.5% chain hotels and 36.5% independent hotels in its top-5 results - matching the underlying data distribution.

8.2 Detecting bias (Cell 24)

Two metrics on the validation set:

- Top-5 EXPOSURE PARITY: 64.24% of top-5 slots are filled by chain hotels (vs the 63.5% baseline). The gap is +0.73 percentage points - demographic parity is essentially intact.
- PER-GROUP NDCG@5: searches that ended in a chain booking scored 0.41575, vs 0.40520 for searches that ended in an independent booking. The gap is +0.01055, meaning the model serves chain-booking users very slightly better than independent-booking users.

The detected bias is therefore subtle: the model surfaces both groups roughly fairly BY EXPOSURE, but it does a slightly better job of ranking the right hotel into the top-5 when that hotel happens to be a chain. The bias lives in ranking quality, not in slot allocation. This is itself an interesting finding - it illustrates one direction of Kleinberg's impossibility theorem from Lecture 8.

8.3 Mitigation: pre-processing re-weighting (Cell 25)

Lecture 8 lists re-weighting as the canonical pre-processing fairness intervention. We assigned weight 2.0 to rows where prop_brand_bool = 0 (independent) AND relevance > 0 (clicked or booked), and weight 1.0 to all other rows. We retrained LambdaMART with identical hyperparameters.

Metric	Original (1.0x)	Re-weighted (2.0x)
Overall NDCG@5	0.38601	0.36067 (-0.025)
Independent top-5 share	35.76%	53.10%
Chain top-5 share	64.24%	46.90%
NDCG@5 independent cohort	0.40520	0.49409
NDCG@5 chain cohort	0.41575	0.32722
Per-group NDCG gap	+0.01055	-0.16687

The 2.0x boost OVER-CORRECTED dramatically: the original +0.011 gap in favour of chain users flipped to a -0.167 gap in favour of independent users - sixteen times larger than the bias we started with.

8.4 Calibration retry: 1.3x (Cell 27)

We retried at 1.3x to find a less aggressive operating point. Overall NDCG@5 dropped to 0.38278 (only 0.003 below original - good on the utility side), and the per-group gap moved to -0.048. Better, but still in

over-corrected territory.

The honest take-away is that the original bias was small enough that any re-weighting strong enough to move the metric ended up overshooting. The fairness-utility frontier is steep around small baseline biases - a finding consistent with Lecture 8's broader Kleinberg-impossibility intuition.

Pre-processing re-weighting can be a blunt instrument when the baseline imbalance is already small. A more surgical method (per-search post-processing with a small group-conditional score offset, for instance) might be a better choice in this regime. We flag it as future work.

9. Deployment Discussion

The assignment asks for a brief discussion of scalable deployment. Four observations:

- Preprocessing as code. Our Task 3 pipeline is fully captured in code (Cells 8 to 16) and the threshold constants and per-property aggregates lookup are pickled to disk. In production, the same script would run on each batch of new search-result data, applying the exact same transforms. The task3_preprocessing.pkl artefact is the canonical handoff: a model server need only read it, apply the documented transforms, and call ranker.predict(...) to score a new batch.
- Model size and inference cost. Our trained LambdaMART forest has 392 trees and serialises to under 20 MB. Inference on 4.96M rows took 22 seconds on a laptop (roughly 225,000 rows per second per CPU). At production query rates (a typical search returns ~25 hotels), this implies ~9,000 search queries per second per CPU.
- Bias monitoring. The Section 8 fairness metrics (top-5 exposure share and per-group NDCG@5) would be straightforward to log continuously in production. A nightly job could compute both on the previous day's traffic and alert when either crosses a configurable threshold.
- Cold-start handling. The notebook's Cell 22 documents the cold-start fallback (filling per-property aggregates with global means for hotels never seen in training). The number of cold-start hotels in production would be small since the catalogue is largely stable month to month.

10. What We Learned

Three things stood out across the project.

GROUP-AWARE SPLITTING MATTERS MORE THAN WE EXPECTED. Our first instinct was a row-level random split; Lecture 7's discussion of per-query evaluation pushed us to GroupShuffleSplit. The 0.001 gap between our local validation NDCG@5 and the public-leaderboard score is the payoff: the metric we optimised against locally is essentially the metric Kaggle scored us on. Without the group-aware split we would have been chasing an inflated number that did not transfer to the test set.

PER-SEARCH RELATIVE FEATURES ARE THE HIGHEST-LEVERAGE FEATURE-ENGINEERING STEP. The relative-rank features we built in Section 5.4 took less than an hour to design and add, and they ended up as four of the top ten features in the final model. The lesson is that for ranking tasks, the right unit of analysis is the search query, not the individual row.

FAIRNESS INTERVENTIONS NEED CALIBRATION. Our re-weighting experiment in Section 8 was sobering. Both the 2.0x and 1.3x boosts over-corrected the original mild bias, and the result was not a bug in our implementation but a real property of the fairness-utility frontier in low-bias regimes. Lecture 8 warned us about this; we now have empirical evidence of what it actually looks like.

THINGS WE WOULD DO DIFFERENTLY. We deferred a per-property historical booking-rate aggregate (Section 5.5 mentions this) to avoid a leakage subtlety; in hindsight, computing it correctly on the training slice would have been worth the extra plumbing and would likely have lifted NDCG@5 by another 0.01-0.02. We also did not explore ensembling - Lecture 7 noted that the published ICDM 2013 winners ensembled multiple gradient-boosted trees with different seeds. A simple two-model average would likely have lifted our Kaggle score by 0.01-0.02 with no risk of overfitting.

DIFFICULTIES WE DID NOT EXPECT. The non-idempotency bug in our missing-value imputation (Section 5.2) was the single most time-consuming issue we ran into. Designing data-preparation cells to be safely re-executable is harder than it sounds; we now write idempotent cells by default. Verifying the Kaggle submission format was also a small surprise: the PDF example shows SearchId, PropertyId as the column header but the live Kaggle grader requires lowercase srch_id, prop_id. Our first submission was rejected because of this; we caught it and resubmitted with the correct header within minutes.

11. Conclusion

We built a LambdaMART learning-to-rank model for the Expedia hotel-booking-prediction task that reached NDCG@5 = 0.38694 on the Kaggle public leaderboard, against a Ridge regression baseline of 0.32414. The pipeline includes 45 engineered features across price transformations, missing-value handling with sentinel-plus-indicator encoding, time features, per-search relative rankings, per-property aggregates, and a competitor-summary collapse. We followed the CRISP-DM process model throughout, ran an iterative two-pass EDA, used a group-aware train/validation split to prevent leakage, and verified our evaluation metric independently of the LightGBM internal computation.

For Task 5, we investigated bias on the chain-vs-independent axis. The model achieves near-perfect demographic parity on top-5 exposure but a small +0.011 NDCG@5 per-group gap in favour of chain users - empirically illustrating one direction of Kleinberg's impossibility theorem from Lecture 8. Pre-processing re-weighting at 2.0x and 1.3x both over-corrected the gap, demonstrating that the fairness-utility frontier is steep in low-bias regimes.

The full code and reproduction instructions are in the accompanying notebook assignment_2_code.ipynb. The artefacts that close out Task 3 (task3_train_features.pkl, task3_preprocessing.pkl, task3_split.npz) and the Kaggle submission (submission.csv) are described in the process report.